# UNIT-1

# INTRODUTION TO PYTHON

**Introduction:** Features of Python, History of Python, installing Python; basic syntax, interactive shell, editing, saving, and running a script. Statements and Expressions, Variables, Operators, Precedence and Associativity, Data Types (Numbers, Booleans, Strings, None), Indentation and comments, Type Conversions, type() Function, membership operator

## Introduction:

Python is a general-purpose, dynamically typed, high-level, compiled and interpreted, garbage-collected, and purely object-oriented programming language that supports procedural, object-oriented, and functional programming.

- It was created by Guido van Rossum, and released in 1991 at National Research Institute for Mathematics and Computer Science in the Netherlands.
- It is derived from programming languages such as ABC, Modula 3, small talk, Algol-68.
- It is Open Source Scripting language.
- It is **Case-sensitive language** (Difference between uppercase and lowercase letters).
- One of the official languages at Google.

Python provides number of libraries and frameworks. Python has gained popularity because of its simplicity, easy syntax and user-friendly environment. The usage of Python as follows.

- Desktop Applications
- Web Applications
- Data Science
- Artificial Intelligence
- Machine Learning
- Scientific Computing
- Robotics
- Internet of Things (IoT)
- Gaming
- Mobile Apps
- Data Analysis and Preprocessing

**Characteristics of Python:**

✓ Interpreted: Python source code is compiled to byte code as a **.pyc** file, and this bytecode can be interpreted by the interpreter.

✓ Interactive

✓ Object Oriented Programming Language

✓ Easy & Simple

✓ Portable

✓ Scalable: Provides improved structure for supporting large programs.

✓ Integrated

✓ Expressive Language

**Python Features**

Python provides many useful features which make it popular and valuable from the other programming languages. It supports object-oriented programming, procedural programming approaches and provides dynamic memory allocation. We have listed below a few essential features.

**1) Easy to Learn and Use**

Python is easy to learn as compared to other programming languages. Its syntax is straightforward and much the same as the English language. There is no use of the semicolon or curly-bracket, the indentation defines the code block. It is the recommended programming language for beginners.

**2) Expressive Language**

Python can perform complex tasks using a few lines of code. A simple example, the hello world program you simply type **print("Hello World")**. It will take only one line to execute, while Java or C takes multiple lines.

**3) Interpreted Language**

Python is an interpreted language; it means the Python program is executed one line at a time. The advantage of being interpreted language, it makes debugging easy and portable.

**4) Cross-platform Language**

Python can run equally on different platforms such as Windows, Linux, UNIX, and Macintosh, etc. So, we can say that Python is a portable language. It enables programmers to develop the software for several competing platforms by writing a program only once.

**5) Free and Open Source**

Python is freely available for everyone. It is freely available on its official website www.python.org. It has a large community across the world that is dedicatedly working towards make new python

modules and functions. Anyone can contribute to the Python community. The open-source means, "Anyone can download its source code without paying any penny."

**6) Object-Oriented Language**

Python supports object-oriented language and concepts of classes and objects come into existence. It supports inheritance, polymorphism, and encapsulation, etc. The object-oriented procedure helps to programmer to write reusable code and develop applications in less code.

**7) Extensible**

It implies that other languages such as C/C++ can be used to compile the code and thus it can be used further in our Python code. It converts the program into byte code, and any platform can use that byte code.

**8) Large Standard Library**

It provides a vast range of libraries for the various fields such as machine learning, web developer, and also for the scripting. There are various machine learning libraries, such as Tensor flow, Pandas, Numpy, Keras, and Pytorch, etc. Django, flask, pyramids are the popular framework for Python web development.

**9) GUI Programming Support**

Graphical User Interface is used for the developing Desktop application. PyQT5, Tkinter, Kivy are the libraries which are used for developing the web application.

**10) Integrated**

It can be easily integrated with languages like C, C++, and JAVA, etc. Python runs code line by line like C,C++ Java. It makes easy to debug the code.

**11. Embeddable**

The code of the other programming language can use in the Python source code. We can use Python source code in another programming language as well. It can embed other language into our code.

**12. Dynamic Memory Allocation**

In Python, we don't need to specify the data-type of the variable. When we assign some value to the variable, it automatically allocates the memory to the variable at run time. Suppose we are assigned integer value 15 to **x,** then we don't need to write **int x = 15.** Just write x = 15.

**Python Version List**

Python programming language is being updated regularly with new features and supports. There are lots of update in Python versions, started from 1994 to current release.

A list of Python versions with its released date is given below.

| Python Version | Released Date |
|---|---|
| Python 1.0 | January 1994 |
| Python 1.5 | December 31, 1997 |
| Python 1.6 | September 5, 2000 |
| Python 2.0 | October 16, 2000 |
| Python 2.1 | April 17, 2001 |
| Python 2.2 | December 21, 2001 |
| Python 2.3 | July 29, 2003 |
| Python 2.4 | November 30, 2004 |
| Python 2.5 | September 19, 2006 |
| Python 2.6 | October 1, 2008 |
| Python 2.7 | July 3, 2010 |
| Python 3.0 | December 3, 2008 |
| Python 3.1 | June 27, 2009 |
| Python 3.2 | February 20, 2011 |
| Python 3.3 | September 29, 2012 |
| Python 3.4 | March 16, 2014 |
| Python 3.5 | September 13, 2015 |
| Python 3.6 | December 23, 2016 |
| Python 3.7 | June 27, 2018 |
| Python 3.8 | October 14, 2019 |

**Python Install**

*To check if we have python installed on a Windows PC, search in the start bar for Python or run the following on the Command Line (cmd.exe):*

***C:\Users*Your Name>*python –version***

*To check if we have python installed on a Linux or Mac, then on linux open the command line or on Mac open the Terminal and type:*

***python –version***

*If we find that we do not have Python installed on your computer, then we can download it for free from the following website: https://www.python.org/*

**Python Interpreter:**

Names of some Python interpreters are:

- PyCharm
- Python IDLE
- The Python Bundle
- pyGUI
- Sublime Text etc.

There are **two modes** to use the python interpreter:

i. Interactive Mode

ii. Script Mode

**i. Interactive Mode**: Without passing python script file to the interpreter, directlyexecute code to Python (Command line).

Example:

>>>6+3

**Output: 9**

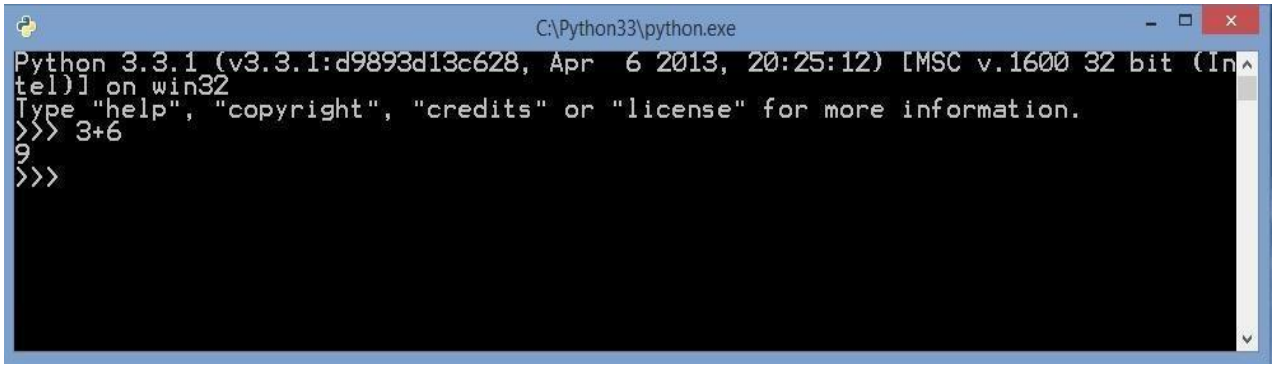



*Fig: Interactive Mode*

***Note:*** >>> is a command (called prompt) the python interpreter uses to indicate thatit is ready. The interactive mode is better when a programmer deals with small pieces of code.

***To run a python file on command line:***

exec(open("C:\Python33\python programs\program1.py").read( ))

**ii. Script Mode:** In this mode source code is stored in a file with the **.py** extension and use the interpreter to execute the contents of the file. To execute the script by the interpreter, we have to tell the interpreter the name of the file.

**Example: if we have a file name** Demo.py **, to run the script we have to follow the following steps:**

**Step-1:** Open the text editor i.e. Notepad

**Step-2:** Write the python code and save the file with .py file extension. (Default directory is *C:\Python33/Demo.py*)

**Step-3:** Open Integrated Development and Learning Environment (IDLE Python GUI) python shell

**Step-4:** Click on file menu and select the open option

**Step-5:** Select the existing python file

**Step-6:** Now a window of python file will be opened

**Step-7:** Click on Run menu and the option Run Module.

**Step-8:** Output will be displayed on python shell window.

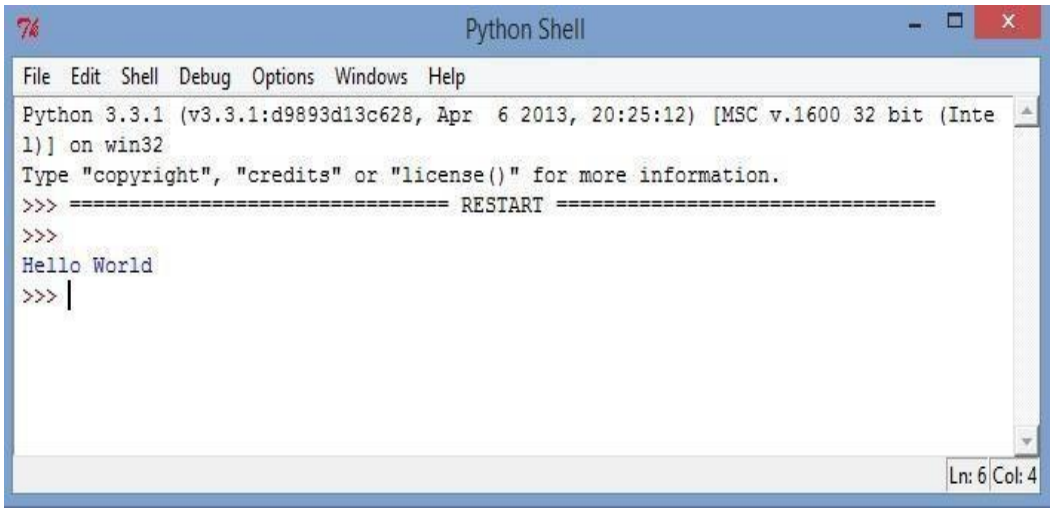



*Fig: Python Shell*

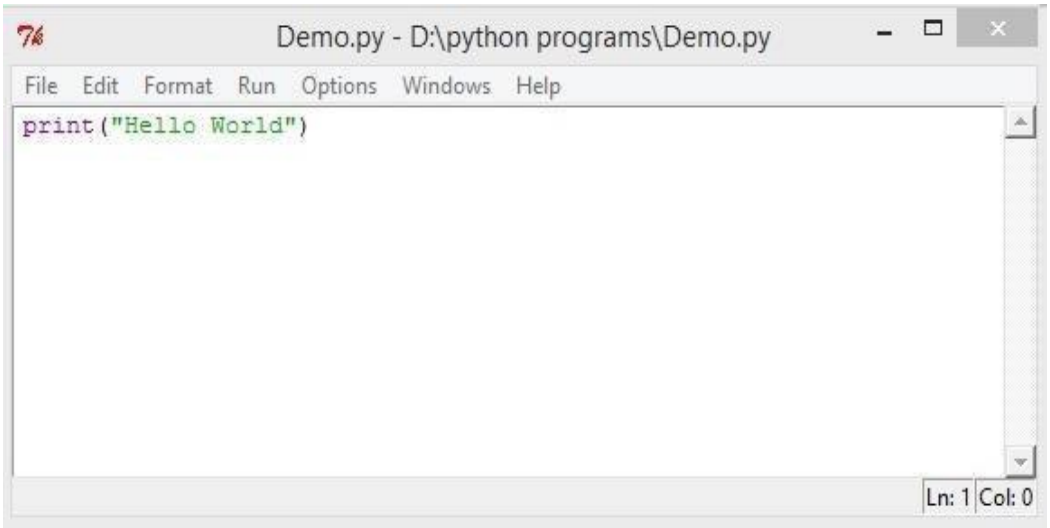



*Fig. : IDLE (Python GUI)*

Whenever we are done in the python command line, we can simply type the following to quit the python command line interface:

```
exit()
```

**Python Character Set** :

It is a set of valid characters that a language recognize.

Letters: A-Z, a-z

Digits : 0-9

Special Symbols

Whitespace

**TOKENS**

**Token**: Smallest individual unit in a program is known as token. There are five types of token in python:

1. Keyword
2. Identifier
3. Literal
4. Operators
5. Punctuators

1. **Keyword**: Reserved words in the library of a language. There are 33 keywords inpython.

| | | | | | |
|---|---|---|---|---|---|
| False | class | finally | is | return | break |
| None | continue | for | lambda | try | except |
| True | def | from | nonlocal | while | in |
| and | del | global | not | with | raise |
| as | elif | if | or | yield | |
| assert | else | import | pass | | |

All the keywords are in lowercase except 03 keywords (True, False, None).

**import keyword**

# printing all keywords at once using "kwlist()"

**print("The list of keywords is : ")**

**print(keyword.kwlist)**

**Output:**

The list of keywords are:

['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break',

'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if',

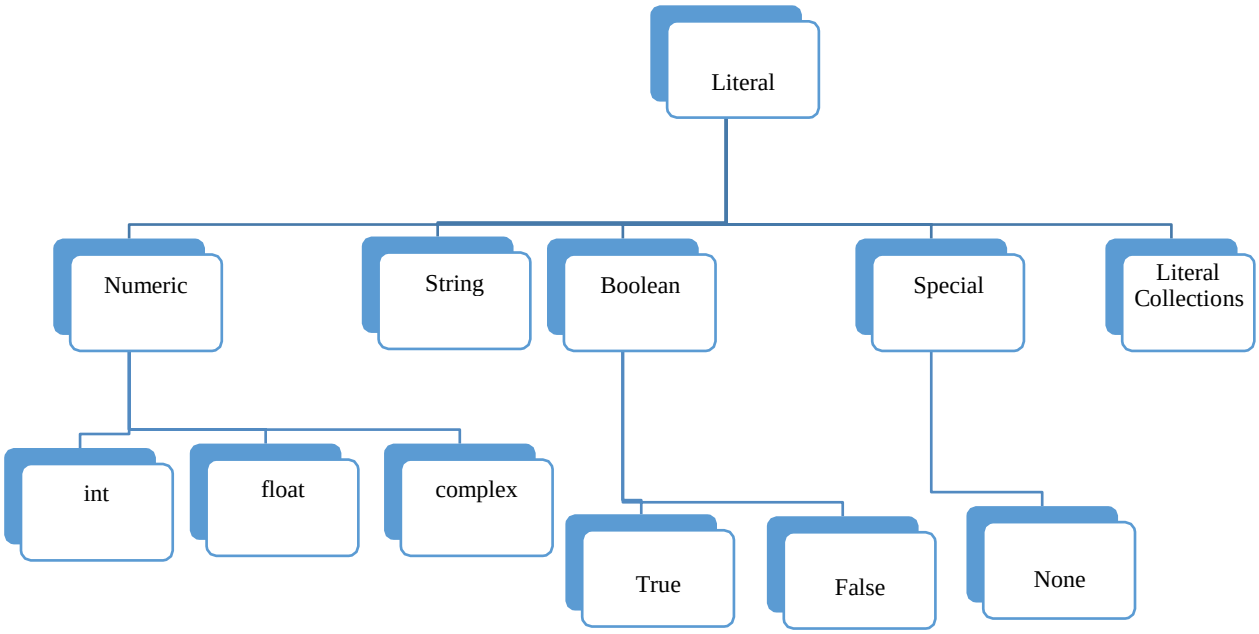
'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']

**2. Identifier:** The name given by the user to the entities like variable name, class-name,function-name etc.

***Rules for identifiers:***

- It can be a combination of letters in lowercase (a to z) or uppercase (A to Z) ordigits (0 to 9) or an underscore.
- It cannot start with a digit.
- Keywords cannot be used as an identifier.
- We cannot use special symbols like !, @, #, $, %, + etc. in identifier.
- _ (underscore) can be used in identifier.
- Commas or blank spaces are not allowed within an identifier.

2. **Literal:** Literals are the constant value. Literals can be defined as a data that is given ina variable or constant.




**Numeric literals**: Numeric Literals are immutable.

**Eg.**

5, 6.7, 6+9j

**A. String literals:**

String literals can be formed by enclosing a text in the quotes. We can use both single aswell as double quotes for a String.

**Eg: "Aman" , '12345'**

**Escape sequence characters:**

| | |
|---|---|
| \\ | Backslash |
| \‘ | Single quote |
| \” | Double quote |
| \a | ASCII Bell |
| \b | Backspace |
| \f | ASCII Formfeed |
| \n | New line charater |
| \t | Horizontal tab |

**B. Boolean literal:** A Boolean literal can have any of the two values: **True** or **False**.

**C. Special literals:** Python contains one special literal i.e. **None**.

**None** is used to specify to that field that is not created. It is also used for end of lists inPython.

**D. Literal Collections:** Collections such as tuples, lists and Dictionary are used in Python.

3. **Operators:** An operator performs the operation on operands. Basically there are two types of operators in python according to number of operands:

**A. Unary Operator:** Performs the operation on one operand.

Example:

| | |
|---|---|
| + | Unary plus |
| - | Unary minus |
| ~ | Bitwise complementnot Logical negation |

**B. Binary Operator**: Performs operation on two operands.

**4. Separator or punctuator : , ; , ( ), { }, [ ]**

**Mantissa and Exponent Form:**

A real number in exponent form has two parts:

- mantissa
- exponent

**Mantissa** : It must be either an integer or a proper real constant.

**Exponent** : It must be an integer. Represented by a letter E or e followed by integer value.

| Valid Exponent form | Invalid Exponent form |
|---|---|
| 123E05 | 2.3E (No digit specified for exponent) |
| 1.23E07 | 0.24E3.2 (Exponent cannot have fractional part) |
| 0.123E08 | 23,455E03 (No comma allowed) |
| 123.0E08 | |
| 123E+8 | |
| 1230E04 | |
| -0.123E-3163.E4 | |
| .34E-24.E3 | |

**Basic terms of a Python Programs:**

A. Blocks and Indentation

B. Statements

C. Expressions

D. Comments

**A. Blocks and Indentation:**

- Python provides no braces to indicate blocks of code for class and function definition or flow control.
- Blocks of code are denoted by line indentation, which is rigidly enforced.
- The number of spaces in the indentation is variable, but all statements within the blockmust be indented the same amount.

**for example** –if True:

```
        print("True")
else:
        print("False")
```

**B. Statements**

**C. Expressions:**

A line which has the instructions or expressions.

A legal combination of symbols and values that produce a result. Generally it produces a value.

**D. Comments:** Comments are not executed. Comments explain a program and make a program understandable and readable. All characters after the # and up to the end of the physical line are part of the comment and the Python interpreter ignores them.

There are two types of comments in python:

i. Single line comment

ii. Multi-line comment

i. **Single line comment**: This type of comments start in a line and when a line ends, it is automatically ends. Single line comment starts with # symbol.

Example: if a>b: # Relational operator compare two values

ii. **Multi-Line comment**: Multiline comments can be written in more than one lines. Triple quoted „ ‟ ‟ or “ ” ”) multi-line comments may be used in python. It is also known as **docstring**.

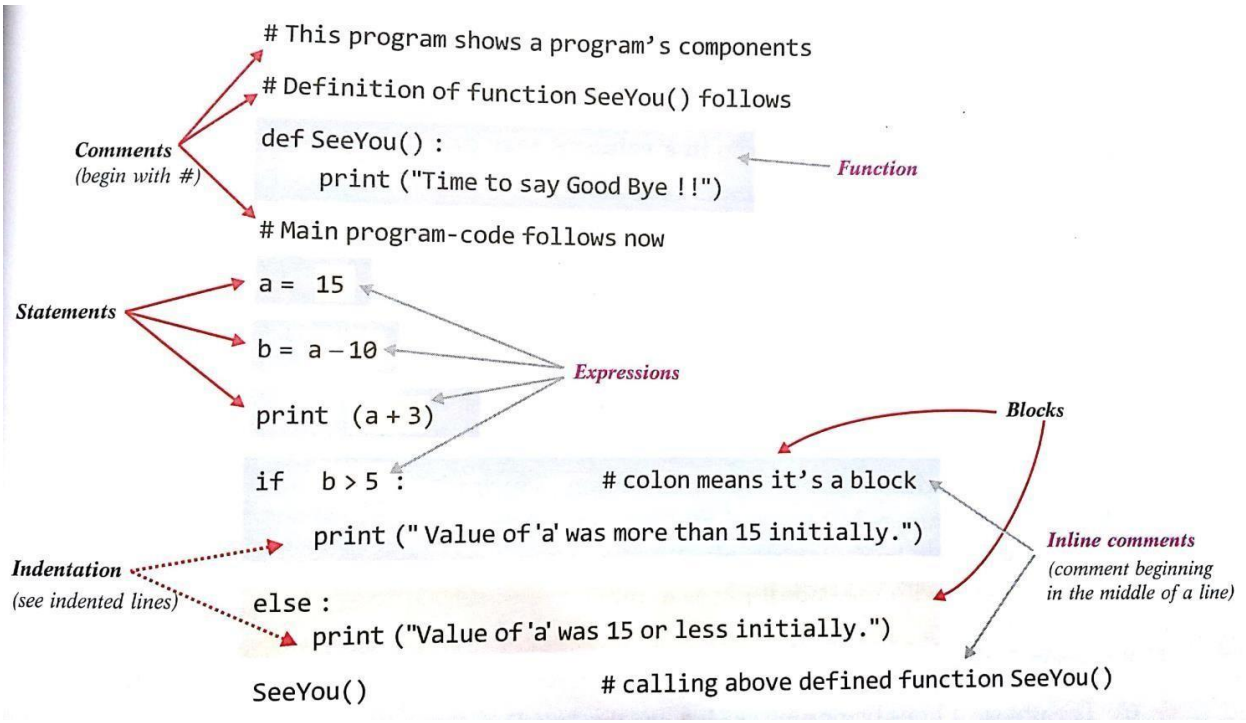
Example:

‘’’ This program will calculate the average of 10 values.

First find the sum of 10 values

and divide the sum by number of values

‘’’




**Multiple Statements on a Single Line:**

The semicolon ( ; ) allows multiple statements on the single line given that neither statementstarts a new code block.

***Example:-***

x=5; print(“Value =” x)

## Variable/Label in Python:

**Definition**: Named location that refers to a value and whose value can be used and processed during program execution.

Variables in python do not have fixed locations. The location they refer to changes every time their values change.

**Creating a variable:**

A variable is created the moment we first assign a value to it.Example:

x = 5

y = “hello”

Variables do not need to be declared with any particular type and can even change type after they have been set. It is known as dynamic Typing.

```
x = 4 # x is of type int
x = "python" # x is now of type str
print (x)
```

**Rules for Python variables:**

- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscore (A-z, 0-9,and _ )
- Variable names are case-sensitive (age, Age and AGE are three different variables)

Python allows assigning a single value to multiple variables.

Example: x = y = z = 5

We can also assign multiple values to multiple variables.

Example: x , y , z = 4, 5, “python”

4 is assigned to x, 5 is assigned to y and string “python” assigned to variable z respectively.

x=12

y=14

x,y=y,x print(x,y)

Now the result will be14 12

**Lvalue and Rvalue:**

An expression has two values. Lvalue and Rvalue.Lvalue: the LHS part of the expression

Rvalue: the RHS part of the expression

Python first evaluates the RHS expression and then assigns to LHS.Example:

p, q, r= 5, 10, 7

q, r, p = p+1, q+2, r-1print (p,q,r)

Now the result will be:

6 6 12

Note: Expressions separated with commas are evaluated from left to right and assigned in sameorder.

❖ If we want to know the type of variable, we can use type( ) function :

**Syntax:**

type (variable-name)

**Example:**

x=6 type(x)

*The result will be:*

<class “int”>

❖ If we want to know the memory address or location of the object, we can use **id( )** function.

**Example:**

>>>id(5) 1561184448

>>>b=5

>>>id(b) 1561184448

We can delete single or multiple variables by using del statement.

Example:del x

del y, z

**Input from a user:**

**input( )** method is used to take input from the user.

Example:

```
print("Enter your name:")
x = input( )
print("Hello, " + x)
```

- input( ) function always returns a value of string type.

**Type Casting:**

To convert one data type into another data type.

Casting in python is therefore done using constructor functions:

**int( )** - constructs an integer number from an integer literal, a float literal or a stringliteral.

Example:

```
x = int(1)          # x will be 1
y = int(2.8)        # y will be 2
z = int("3")        # z will be 3
```

**float( )** - constructs a float number from an integer literal, a float literal or a string literal.

Example:

```
x = float(1)        # x will be 1.0
y = float(2.8)      # y will be 2.8
z = float("3")      # z will be 3.0
w = float("4.2")    # w will be 4.2
```

**complex()** - constructs a complex numbers are written with a "j" as the imaginary part:

```
x = 3+5j            # x will be (3+5j)
y = 5j              # y will be 5j
z = -5j             #z will be (-0-5j)
a = 1               # int
b = complex(a)      #b will be (1+0j)
```

**str( )** - constructs a string from a wide variety of data types, including strings, integerliterals and float literals.

Example:

```
x = str("s1")          # x will be 's1'
y = str(2)             # y will be '2'
z = str(3.0)           # z will be '3.0'
```

**Reading a number from a user:**

x= int (input("Enter an integer number"))

OUTPUT using print( ) statement:

**Syntax:**

print(object, sep=<separator string >, end=<end-string>)

**object** : It can be one or multiple objects separated by comma.

**sep** : sep argument specifies the separator character or string. It separate the objects/items. Bydefault sep argument adds space in between the items when printing.

**end** : It determines the end character that will be printed at the end of print line. By default it has newline character( „\n" ).

**Example**:

x=10 y=20 z=30

print(x,y,z, sep="@", end= „ „)

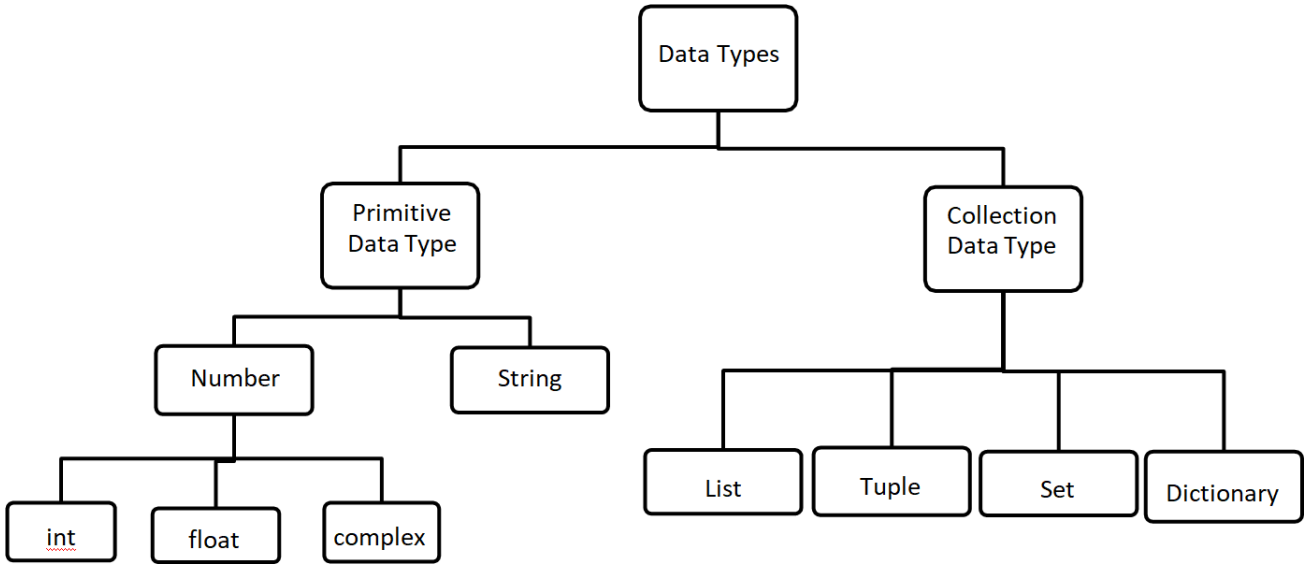
**Output**:

10@20@30

**Data Types in Python:**

Python has Two data types:

1. Primitive Data Type (Numbers, String)
2. Collection Data Type (List, Tuple, Set, Dictionary)




1. **Primitive Data Types:**

a. **Numbers**: Number data types store numeric values.

There are three numeric types in Python:

- int
- float
- complex

Example:

```
w = 1          # int
y = 2.8        # float
z = 1j         # complex
```

- **integer** : There are two types of integers in python:
- int
- Boolean

 **int:** int or integer, is a whole number, positive or negative, without decimals.

*Example:*

x = 1

y = 35656222554887711

z = -3255522

 **Boolean:** It has two values: True and False. **True** has the value **1** and **False** has thevalue **0**.

*Example*:

>>>bool(0)

**False**

>>>bool(1)

**True**

>>>bool(„ „)

**False**

>>>bool(-34)

**True**

>>>bool(34)

**True**

 **float** : float or "floating point number" is a number, positive or negative, containing oneor more decimals. Float can also be scientific numbers with an "e" to indicate the powerof 10.

*Example:*

x = 1.10

y = 1.0

z = -35.59

a = 35e3b = 12E4

c = -87.7e100

 **complex** : Complex numbers are written with a "j" as the imaginary part.

*Example:*

>>>x = 3+5j

>>>y = 2+4j

>>>z=x+y

>>>print(z) 5+9j

>>>z.real 5.0

>>>z.imag 9.0

Real and imaginary part of a number can be accessed through the attributes **real** and **imag**.

**b. String:** Sequence of characters represented in the quotation marks.

- Python allows for either pairs of single or double quotes. Example: 'hello' is the sameas "hello" .
- Python does not have a character data type, a single character is simply a string with alength of **1**.
- The python string store Unicode characters.
- Each character in a string has its own index.
- String is immutable data type means it can never change its value in place.

**2. Collection Data Type:**

- List
- Tuple
- Set
- Dictionary

***Python Operators***

Operators are used to perform operations on variables and values.

In the example below, we use the + operator to add together two values:

Example: print(10 + 5)

**Python divides the operators in the following groups:**

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Identity operators
- Membership operators
- Bitwise operators

### *Python Arithmetic Operators*

Arithmetic operators are used with numeric values to perform common mathematical operations:

| Operator | Name | Example |
|---|---|---|
| + | Addition | x + y |
| - | Subtraction | x - y |
| * | Multiplication | x * y |
| / | Division | x / y |
| % | Modulus | x % y |
| ** | Exponentiation | x ** y |
| // | Floor division | x // y |

### *Python Assignment Operators*

Assignment operators are used to assign values to variables:

| Operator | Example | Same As |
|---|---|---|
| = | x = 5 | x = 5 |
| += | x += 3 | x = x + 3 |
| -= | x -= 3 | x = x - 3 |
| *= | x *= 3 | x = x * 3 |
| /= | x /= 3 | x = x / 3 |
| %= | x %= 3 | x = x % 3 |
| //= | x //= 3 | x = x // 3 |
| **= | x **= 3 | x = x ** 3 |
| &= | x &= 3 | x = x & 3 |
| \|= | x \|= 3 | x = x \| 3 |
| ^= | x ^= 3 | x = x ^ 3 |
| >>= | x >>= 3 | x = x >> 3 |
| <<= | x <<= 3 | x = x << 3 |

### *Python Comparison Operators*

Comparison operators are used to compare two values:

| Operator | Name | Example |
|---|---|---|
| == | Equal | x == y |

| | | |
|---|---|---|
| != | Not equal | x != y |
| > | Greater than | x > y |
| < | Less than | x < y |
| >= | Greater than or equal to | x >= y |
| <= | Less than or equal to | x <= y |

***Python Logical Operators***

Logical operators are used to combine conditional statements:

| **Operator** | **Description** | **Example** |
|---|---|---|
| and | Returns True if both statements are true | x < 5 and x < 10 |
| or | Returns True if one of the statements is true | x < 5 or x < 4 |
| not | Reverse the result, returns False if the result is true | not(x < 5 and x < 10) |

***Python Identity Operators***

Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location:

| **Operator** | **Description** | **Example** |
|---|---|---|
| is | Returns True if both variables are the same object | x is y |
| is not | Returns True if both variables are not the same object | x is not y |

***Python Membership Operators***

Membership operators are used to test if a sequence is presented in an object:

| **Operator** | **Description** | **Example** |
|---|---|---|
| in | Returns True if a sequence with the specified value is present in the object | x in y |
| not in | Returns True if a sequence with the specified value is not present in the object | x not in y |

***Python Bitwise Operators***

Bitwise operators are used to compare (binary) numbers:

| Operator | Name | Description | Example |
|---|---|---|---|
| & | AND | Sets each bit to 1 if both bits are 1 | x & y |
| \| | OR | Sets each bit to 1 if one of two bits is 1 | x \| y |
| ^ | XOR | Sets each bit to 1 if only one of two bits is 1 | x ^ y |
| ~ | NOT | Inverts all the bits | ~x |
| << | Zero fill left shift | Shift left by pushing zeros in from the right and let the leftmost bits fall off | x << 2 |
| >> | Signed right shift | Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off | x >> 2 |

***Operator Precedence***

Operator precedence describes the order in which operations are performed.

**Example**

Parentheses has the highest precedence, meaning that expressions inside parentheses must be evaluated first:

```
print((6 + 3) - (6 + 3))
```

**Example**

Multiplication * has higher precedence than addition +, and therefor multiplications are evaluated before additions:

```
print(100 + 5 * 3)
```

The precedence order is described in the table below, starting with the highest precedence at the top:

| Operator | Description |
|---|---|
| () | Parentheses |
| ** | Exponentiation |
| +x -x ~x | Unary plus, unary minus, and bitwise NOT |
| * / // % | Multiplication, division, floor division, and modulus |
| + - | Addition and subtraction |

| | |
|---|---|
| << >> | Bitwise left and right shifts |
| & | Bitwise AND |
| ^ | Bitwise XOR |
| \| | Bitwise OR |
| == != > >= < <= is is not in not in | Comparisons, identity, and membership operators |
| not | Logical NOT |
| and | AND |
| or | OR |
| = | Assignment |

If two operators have the same precedence, the expression is evaluated from left to right.

***Mutable and Immutable Data Types in Python***

Mutable or immutable is the fancy word for explaining the property of data types of being able to get updated after being initialized. The basic explanation is thus: A mutable object is one whose internal state is changeable. On the contrary, once an immutable object in Python has been created, we cannot change it in any way.

**What are Mutable Data Types?**

Anything is said to be mutable when anything can be modified or changed. The term "mutable" in Python refers to an object's capacity to modify its values. These are frequently the things that hold a data collection.

**What are Immutable Data Types?**

Immutable refers to a state in which no change can occur over time. A Python object is referred to as immutable if we cannot change its value over time. The value of these Python objects is fixed once they are made.

**List of Mutable and Immutable objects**

Python mutable data types:

- Lists
- Dictionaries
- Sets
- User-Defined Classes (It depends on the user to define the characteristics of the classes)

Python immutable data types:

- Numbers (Integer, Float, Complex, Decimal, Rational & Booleans)
- Tuples
- Strings
- Frozen Sets

**The Python id() Function**

When you define a Python object, the program sets a memory section aside. Every Python object has a distinct address that informs the application where the item is located in memory.

Every object in Python has a distinct ID connected to the object's memory address. Use the built-in Python id() function to read the special ID.

Let's read the position of a sample string object in memory, for instance:

**Code**

```
# Python program to show how to use the id function

# Initializing a string object
string = "string"

# Printing the id of the string object
print(id(string))
```

**Output:**

```
140452604995952
```

**Mutability in Python**

As we have already told, an immutable Python object cannot be altered.

In Python, the integer data type is a prime case of an immutable object or data type. Let's conduct an easy experiment to understand this.

Let's first make two integer variables, x and y. The variable y is a reference to the variable x:

Now, x and y both point towards the same location in memory. In other words, both integer variables should have the same ID returned by the id() function. Let's confirm that this is true:

**Code**

```
# Python program to create two variables having the same memory reference

# Initializing a variable
x = 3

# Storing it in another variable
y = x

# Checking if two have the same id reference
print("x and y have same id: ", id(x) == id(y))
```

**Output:**

```
x and y have same id:  True
```

As a result, the variables x and y points to a single integer object. In other words, even though there is just one integer variable, two variables refer to it.

Let us change x's value now. Next, let's compare the reference ids of x and y once more:

**Code**

```
# Python program to check if x and y have the same ids after changing the value

x = 3
y = x

# Changing the value of x
x = 13

# Checking if x and y still point to the same memory location
print("x and y have the same ids: ", id(x) == id(y))
```

**Output:**

```
x and y have the same ids: False
```

This is because x now points to a distinct integer object. Therefore, the integer variable 3 itself remained constant. However, the variable x that previously referred to it now directs users to the new integer entity 13.

Therefore, even though it appears like we modified the original integer variable, we didn't. In Python, the integer object is an immutable data type, meaning that an integer object cannot be changed once created.

Let's conduct a similar test once more using a mutable object.
Create a list, for instance, and put it in a second variable. Then, compare the reference IDs of the two lists. Let's then make some changes to the list we created. Then, let's see if the reference IDs of both lists still coincide:

**Code**

```
1.  # Python program to check if the lists pointing to the same memory location will have the same reference ids after modifying one of the lists
2.
3.  # Creating a Python list object
4.  num = [2, 4, 6, 8]
5.
6.  # Storing the list in another variable
7.  l = num
8.
9.  # Checking if the two variables point to the same memory location
10. print("Both list variables have the same ids: ", id(num) == id(l))
11.
12. # Modifying the num list
13. num.append(10)
14.
15. # Checking if, after modifying num, both the lists point to the same memory location
16. print("Both list variables have the same ids: ", id(num) == id(l))
```

**Output:**

```
Both list variables have the same ids:  True
Both list variables have the same ids:  True
```

Furthermore, their reference IDs match. As a result, the list objects num and l point to the memory location. This demonstrates that we could explicitly change the list object by adding one more element. The list data structures must therefore be mutable. And Python lists operate in this way.

***Example of Mutable Objects in Python***

**List**

As a result of their mutable nature, lists can change their contents by incorporating the assignment operators or the indexing operators.

**Code**

```
# Python program to show that a list is a mutable data type

# Creating a list
list1 = ['Python', 'Java', 23, False, 5.3]
print("The original list: ", list1)

# Changing the value at index 2 of the list
list1[2]='changed'
print("The modified list: ", list1)
```

**Output:**

```
The original list:  ['Python', 'Java', 23, False, 5.3]
The modified list:  ['Python', 'Java', 'changed', False, 5.3]
```

**Dictionary**

Due to the mutability of dictionaries, we can modify them by implementing a built-in function update or using keys as an index.

Let's look at an illustration of that.

**Code**

```
# Python program to show that a dictionary is a mutable data type

# Creating a dictionary
dict_ = {1: "a", 2: "b", 3: "c"}
print("The original dictionary: ", dict_)
```

```
6.
7.  # Changing the value of one of the keys of the dictionary
8.  dict_[2]= 'changed'
9.  print("The modified dictionary: ", dict_)
```

**Output:**

```
The original dictionary:  {1: 'a', 2: 'b', 3: 'c'}
The modified dictionary:  {1: 'a', 2: 'changed', 3: 'c'}
```

**Set**

Due to the mutability of sets, we can modify them using a built-in function (update).

See an illustration of that:

**Code**

```
1.  # Python program to show that a set is a mutable data type
2.
3.  # Creating a set
4.  set_ = {1, 2, 3, 4}
5.  print("The original set: ", set_)
6.
7.  # Updating the set using the update function
8.  update_set = {'a', 'b', 'c'}
9.  set_.update(update_set)
10. print("The modified set: ", set_)
```

**Output:**

```
The original set:  {1, 2, 3, 4}
The modified set:  {1, 2, 3, 4, 'b', 'a', 'c'}
```

***Example of Immutable Python Objects***

**int**

Since int in Python is an immutable data type, we cannot change or update it.

As we previously learned, immutable objects shift their memory address whenever they are updated.

**Code**

```
1.  # Python program to show that int is an immutable data type
```

```
2.
3.  int_ = 25
4.  print('The memory address of int before updating: ', id(int_))
5.
6.  # Modifying an int object by giving a new value to it
7.  int_ = 35
8.  print('The memory address of int after updating: ', id(int_))
```

**Output:**

```
The memory address of int before updating:  11531680
The memory address of int after updating:  11532000
```

**float**

Since the float object in Python is an immutable data type, we cannot alter or update it. As we previously learned, immutable objects shift their memory address whenever they are updated.

Here is an illustration of that:

**Code**

```
1.  # Python program to show that float is an immutable data type
2.
3.  float_ = float(34.5)
4.  print('The memory address of float before updating: ', id(float_))
5.
6.  # Modifying the float object by giving a new value to it
7.  float_ = float(32.5)
8.  print('The memory address of float after updating: ', id(float_))
```

**Output:**

```
The memory address of float before updating:  139992739659504
The memory address of float after updating:  139992740128048
```

**String**

Since strings in Python are immutable data structures, we cannot add or edit any data. We encountered warnings stating that strings are not changeable when modifying any section of the string.

Here is an illustration of that:

**Code**

```
1.  # Python program to show that a string is an immutable data type
2.
3.  # Creating a string object
4.  string = 'hello peeps'
5.
6.  # Trying to modify the string object
7.  string[0] = 'X'
8.  print(string)
```

**Output:**

```
TypeError                                 Traceback (most recent call last)
<ipython-input-3-4e0fff91061f> in <module>
      3 string = 'hello peeps'
      4
----> 5 string[0] = 'X'
      6
      7 print(string)

TypeError: 'str' object does not support item assignment
```

**Tuple**

Because tuples in Python are immutable by nature, we are unable to add or modify any of their contents. Here is an illustration of that:

**Code**

```
1.  # Python program to show that a tuple is an immutable data type
2.
3.  # Creating a tuple object
4.  tuple_ = (2, 3, 4, 5)
5.
6.  # Trying to modify the tuple object
7.  tuple_[0] = 'X'
8.  print(tulple_)
```

**Output:**

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-5-e011ebc4971e> in <module>
      5
      6 # Trying to modify the tuple object
----> 7 tuple_[0] = 'X'
      8 print(tulple_)
      9


TypeError: 'tuple' object does not support item assignment
```

**FrozenSet**

Like sets, frozensets are immutable data structures. While the set's components can always be changed, this cannot be done with a frozenset. Therefore, nothing can be added to or updated in the frozenSet.

Here is an illustration of that:

**Code**

```
1.  # Python program to show that a frozenset is an immutable data type
2.
3.  # Creating a frozenset object
4.  t = (2, 4, 5, 7, 8, 9, 5, 9, 6, 11, 12)
5.  fset = frozenset(t)
6.
7.  # Updating the value of the frozenset
8.  fset[1] = 76
9.  print(fset)
```

**Output:**

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-1-2cafcda94faf> in <module>
      6
      7 # Updating the value of the frozenset
----> 8 fset[1] = 76
```

```
 9 print(fset)
10


TypeError: 'frozenset' object does not support item assignment
```

***The random() Function***

The random.random() function gives a float number that ranges from 0.0 to 1.0. There are no parameters required for this function. This method returns the second random floating-point value within [0.0 and 1] is returned.

**Code**

```
1.  # Python program for generating random float number
2.  import random
3.  num=random.random()
4.  print(num)
```

**Output:**

```
0.3232640977876686
```

***The randint() Function***

The random.randint() function generates a random integer from the range of numbers supplied.

**Code**

```
1.  # Python program for generating a random integer
2.  import random
3.  num = random.randint(1, 500)
4.  print( num )
```

**Output:**

```
215
```

***The randrange() Function***

The random.randrange() function selects an item randomly from the given range defined by the start, the stop, and the step parameters. By default, the start is set to 0. Likewise, the step is set to 1 by default.

**Code**

```
1.  # To generate value between a specific range
```

```
2. import random
3. num = random.randrange(1, 10)
4. print( num )
5. num = random.randrange(1, 10, 2)
6. print( num )
```

**Output:**

```
4
9
```

***The choice() Function***

The random.choice() function selects an item from a non-empty series at random. In the given below program, we have defined a string, list and a set. And using the above choice() method, random element is selected.

**Code**

```
1. # To select a random element
2. import random
3. random_s = random.choice('Random Module') #a string
4. print( random_s )
5. random_l = random.choice([23, 54, 765, 23, 45, 45]) #a list
6. print( random_l )
7. random_s = random.choice((12, 64, 23, 54, 34)) #a set
8. print( random_s )
```

**Output:**

```
M
765
54
```

**Apply eval() to input() Function**

We know that the input() function returns every input by the user as string, including numbers. And this problem was solved by making use of type before input() function.

Example X = int (input('Enter the Number'))

Once the above statement is executed, Python returns it into its respective type. By making use of eval() function, we can avoid specifying a particular type in front of input() function.

Thus, the above statement,

X = int (input('Enter the Number'))

can be written as:

X = eval(input('Enter the Number'))

**FORMATTING NUMBER AND STRINGS**

A formatting string helps make string look presentable to the user for printing. A programmer can make use of format function to return a formatted string. Consider the following example to calculate the area of a circle before using this function.

The syntax of format function is : **format(item, format-specifier)**

```
radius = int(input('Please Enter the Radius of Circle: '))
print(' Radius = ', radius)              #Print Radius
PI = 3.1428                              #Initialize value of PI
Area = PI * radius * radius              #Calculate Area
print(' Area of Circle is: ',Area)       #Print Area

Output

Please Enter the Radius of Circle: 4
Radius = 4
Area of Circle is: 50.2848
```

use of format() function and display the area of a circle.

```
radius = int(input('Please Enter the Radius of Circle: '))
print(' Radius = ', radius)
PI = 3.1428
Area = PI * radius * radius
print(' Area of Circle is: ',format(Area,'.2f'))

Output

Please Enter the Radius of Circle: 4
Radius = 4
Area of Circle is: 50.28
```

**Formatting Floating Point Numbers**

print(format(10.345,"10.2f"))

print(format(10,”10.2f”))

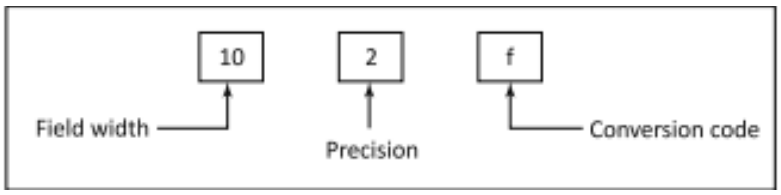
print(format(10.32245,”10.2f”))

displays the output as follows:

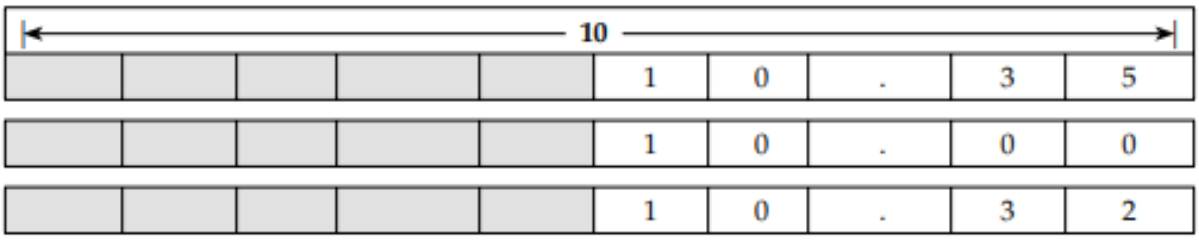
10.35

10.00

10.32




The actual representation of the above output is:



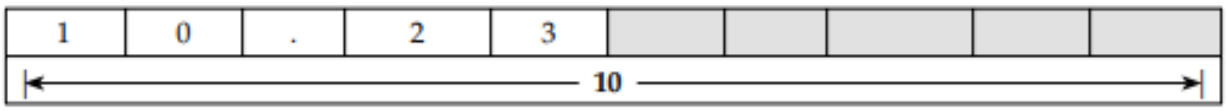

>>>print(format(10.234566,”10.2f”)) #Right Justification

10.23

>>> print(format(10.234566 ,”<10.2f”)) #Left Justification Example

10.23

The actual representation of the above output for left justification is:




**Integer Formatting**

Example

>>>print(format(20,”10x”)) #Integer formatted to Hexadecimal Integer

14

>>> print(format(20,”<10x”))

14

>>> print(format(1234,”10d”)) #Right Justification

1234

**Formatting String**

A programmer can make use of conversion code s to format a string with a specifi ed width. However, by default, string is left justifi ed. Following are some examples of string formatting.

Example

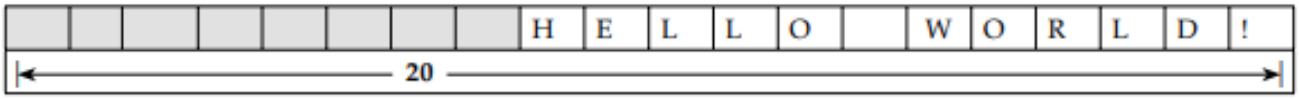
>>> print(format("Hello World!","25s") #Left Justification

Example

Hello World!

>>>print(format("HELLO WORLD!",">20s")) #String Right Justification

HELLO WORLD




**PYTHON INBUILT FUNCTIONS**

| *Function* | *Description* |
|---|---|
| **abs(x)** | Returns absolute value of x |
| **Example**<br>`>>> abs(-2)`<br>`2`<br>`>>> abs(4)`<br>`4` | |

| | |
|---|---|
| **max(x1, X2, X3,……,XN)** | Returns largest value among X1, X2, X3, X4,… , XN |
| **Example:**<br>`>>> max(10,20,30,40)`<br>`40` | |
| **max(x1, X2, X3,……,XN)** | Returns minimum value among X1, X2, X3, X4,… , XN. |
| **pow(X, Y)** | Return the $\mathbf{X^Y}$ |
| **Example:**<br>`>>> pow(2,3)`<br>`8` | |
| **round(x)** | Returns an integer nearest to the value of x. |
| **Example:**<br>`>>> round(10.34)`<br>`10`<br>`>>> round(10.89)`<br>`11` | |

**Inbuilt mathematical functions in Python**

| *Function* | *Example* | *Description* |
|---|---|---|
| `ceil(X)` | `>>> math.ceil(10.23)`<br>`11` | Round X to nearest integer and returns that integer. |
| `floor(X)` | `>>> math.floor(18.9)`<br>`18` | Returns the largest value not greater than X |
| `exp(X)` | `>>> math.exp(1)`<br>`2.718281828459045` | Returns the exponential value for $e^x$ |
| `log(X)` | `>>> math.log(2.71828)`<br>`0.999999327347282` | Returns the natural logarithmic of x (to base e) |
| `log(x,base)` | `>>> math.log(8,2)`<br>`3.0` | Returns the logarithmic of x to the given base |
| `sqrt(X)` | `>>>math.sqrt(9)`<br>`3.0` | Return the square root of x |
| `Sin(X)` | `>>> math.sin(3.14159/2)`<br>`0.9999999999991198` | Return the sin of X, where X is the value in radians |
| `asin(X)` | `>>> math.asin(1)`<br>`1.5707963267948966` | Return the angle in radians for the inverse of sine |
| `cos(X)` | `>>> math.cos(0)`<br>`1.0` | Return the sin of X, where X is the value in radians |
| `aCos(X)` | `>>> math.acos(1)`<br>`0.0` | Return the angle in radians for the inverse of cosine |
| `tan(X)` | `>>> math.tan(3.14/4)`<br>`0.9992039901050427` | Return the tangent of X, where X is the value in radians |
| `degrees(X)` | `>>> math.degrees(1.57)`<br>`89.95437383553924` | Convert angle X from to radians to degrees |
| `Radians(X)` | `>>> math.`<br>`radians(89.99999)`<br>`1.5707961522619713` | Convert angle x from degrees to radians |

**The ord and chr Functions**

The American Standard Code for Information Interchangeable ( ASCII) is one of the most popular encoding schemes. It is a 7-bit encoding scheme for representation of all lower and upper case letters, digits and punctuation marks. The ASCII uses numbers from 0 to 127 to represent all characters. Python uses the in-built function ord(ch) to return the ASCII value for a given character.

Example

>>> ord(‘A’) #Returns ASCII value of Character ‘A’

65

>>> ord(‘Z’) #Returns ASCII Value of Character of ‘Z’

90

>>> ord(‘a’) #Returns ASCII Value of Character of ‘a’

97

>>> ord(‘z’) #Returns ASCII Value of Character of ‘z’

122

The chr(Code) returns the character corresponding to its code, i.e. the ASCII value. The following example demonstrates the use of in-built function chr().

Example

```
>>> chr(90)
'Z'
>>> chr(65)
'A'
>>> chr(97)
'a'
>>> chr(122)
'z'
```